Consent as Infrastructure: Keeping AI Agents Tied to the People They Act For

Preethika Reddy Mandadi, Medifakt

Preprint. This work has been submitted to the IEEE for possible publication. Copyright may be transferred without notice, after which this version may no longer be accessible.

Abstract—AI agents act on delegated authority, yet authorization is granted once and checked against a fixed grant. Drawing on healthcare consent infrastructure, I argue that delegated authority must stay tied to the principal throughout execution: purpose enforced at runtime, scope changeable mid-task, and the audit record visible to the principal.

Keywords—AI agents; authorization; consent; delegation; runtime enforcement; audit; prompt injection

Delegation Is Not a One-Time Event

When a person hands a task to an AI agent today, authorization usually happens once. Permissions are granted at the start, and each later action is checked against that fixed grant. Agent tasks, however, unfold over time. The agent reads untrusted content, spawns sub-agents, and meets situations no one anticipated, while the person's own intent may change.

This article argues that delegated authority should stay connected to the person who granted it for the whole of execution, not only at the moment of grant. I came to this question while building consent infrastructure for a health data platform, where the same problem has a long and instructive history.

The building blocks are not missing. IETF drafts address agent identity, delegated authorization, revocation signals, and tamper-evident audit [1], [2]. Research prototypes bind tool use to a declared task or intent [3], [4]. Agent frameworks ship approval gates. What is unsettled is whom these pieces serve and how they fit together. The drafts now in progress are where that will be decided.

Healthcare is a useful but imperfect reference. Through decades of failure and regulation, it learned to separate recording consent from enforcing it, and to keep a record of what was done with patient data. Those lessons do not transfer wholesale, because agents take actions rather than only reading records. They do, however, sharpen three questions that current specifications leave open:

1. **Purpose.** Should a task's declared purpose constrain what the agent may do at runtime, rather than serve as a label in an audit log?
2. **Mid-task change.** Should the principal be able to change what a running agent may do, as a class of actions? That differs both from approving single held actions and from automatic, risk-triggered attenuation.
3. **Audit.** Should the principal, not only the operator, have meaningful access to the record of what was done?

I argue yes to all three. The contribution is a normative framing for specifications that are still being written. It does not propose new mechanisms; designing mechanisms that meet these requirements is future work.

Terms. The *principal* is the person or organization whose authority the agent exercises. The *operator* deploys and runs the agent. The two may coincide, as when an enterprise runs agents for itself, or diverge, as when a consumer uses an agent deployed by a third party. Some specifications use "principal" for the deploying party [2]; here it always means the party whose authority is delegated.

What Healthcare Built, and What It Didn't

Healthcare's consent infrastructure grew out of failure, not foresight. The Tuskegee Syphilis Study exposed research conducted without informed consent. The case of Henrietta Lacks raised lasting questions about the use of tissue. The sale of patient data showed how little recourse individuals had. Regulation followed through separate channels: the Belmont Report and the Common Rule for research, HIPAA and later HITECH in the United States, and GDPR in Europe.

That regulation is narrower than often assumed. HIPAA permits uses for treatment, payment, and healthcare operations without patient consent [5], and GDPR treats consent as one lawful basis among several [6]. Healthcare is therefore not a model of universal patient control. What it offers is architecture.

Representation is not enforcement. The HL7 FHIR Consent resource records who grants permission, to whom, for what purpose, and for how long [7]. The specification deliberately leaves enforcement out of scope, expecting separate access-control mechanisms such as OAuth, UMA, or XACML to apply the directive. SMART on FHIR adds OAuth-based authorization for applications, but a valid token does not by itself show that every consent directive was evaluated. Production systems that connect these layers, such as consent-aware FHIR gateways, reveal the practical limits: consent changes take time to reach enforcement, and coverage of operations is never complete.

Keep the layers apart. Healthcare's architecture separates four things that agent systems often blur. *Consent* expresses the principal's choices. *Delegation* transfers a bounded part of that authority. *Authorization* turns choices and policy into decisions about specific actions. *Enforcement* applies those decisions at runtime. Keeping these layers separate and legible is the central design lesson this article carries over.

From this experience I draw four recurring properties. They are my synthesis, not a formal healthcare framework:

Table 1. Four properties drawn from healthcare consent infrastructure.

Property	In healthcare	For AI agents
Revocability	Consent can be withdrawn, and the change propagated to enforcement	A principal's revocation takes effect whatever the agent's reasoning
Purpose limitation	GDPR Art. 5(1)(b); HIPAA's minimum-necessary standard [5], [6]	Actions are bounded by the task the principal delegated
Auditability	Access is logged with enough context to reconstruct it	Each action is linked to the authorization decision behind it
Granularity	Permissions are scoped, not all-or-nothing	Oversight scales with an action's sensitivity, not its volume

Before standards such as FHIR and SMART on FHIR, individual hospitals had access controls but no shared framework organized around the patient's consent rather than the institution's policy. Agent authorization is at a similar point today. The mechanisms exist across providers and protocols, but they are not yet organized around the person whose authority they exercise.

Two cautions temper the analogy. First, health-data access events are discrete and usually attributable, while agent actions can be compound and irreversible: an email sent or a payment captured cannot be recalled. Second, healthcare never fully solved its own problem. Consent directives are inconsistently honored, patients rarely see audit trails, and many people sign forms they do not understand. The lesson is not that healthcare got it right. It is that the policy existed for years before infrastructure enforced it, and AI agents may not have that long.

Sidebar: How a consent change travels. In a consent-aware health data system, a patient who withdraws permission for research use updates a FHIR Consent record. A separate policy engine reads that record when an access request arrives and decides whether to allow it; the application holding an access token learns nothing directly. Two practical lessons follow. First, a change is only as fast as the path from record to enforcement: if the engine caches decisions, a withdrawal can lag behind the patient's choice. Second, coverage must be explicit: operations the engine does not intercept are not governed by the consent at all. Both lessons carry over to agents. A principal's mid-task change must reach the enforcement point before the next action, and any tool path the enforcement point does not see is outside the principal's control.

Where Agent Authorization Stands

Most agent safety work shapes the model. Training instills values, and system prompts set boundaries at the start of a session. Those controls matter, but they are fixed before execution begins. When an injected instruction overrides a system prompt, or an agent is deployed in a context its designers never anticipated, nothing independent re-asserts the boundary. Runtime controls are emerging to fill that gap, and recent evidence shows why they are needed.

The enforcement gap is now measured. Khan tested six widely used open-source agent frameworks and found that approval pauses did not reliably stop side effects: with live models driving unmodified frameworks, 215 of 1,200 runs leaked, meaning an action on a parallel branch took effect while an approval gate was supposedly pausing the task [8]. On naturalistic benchmark tasks the gap stayed latent, because models tended to act sequentially, but prompt injection could trigger it deterministically. An external gate between the agent and its tools, SOUNDGATE, blocked every measured violation under its stated mediation contract.

Organizations see the consequences. In a Cloud Security Alliance survey of 418 IT and security professionals, commissioned by Token Security, 82% had found previously unknown agents, 65% reported an agent-related incident in the past year, 21% had formal decommissioning processes, and only 11% automatically blocked agents that exceeded their scope [9].

Standards and research are responding, and each covers part of the ground:

Table 2. What current specifications and research provide, and what they leave open.

Work	What it provides	What it leaves open
AIMS-01, IETF WIMSE working group [1]	Agent identity, OAuth-based delegation, down-scoping on risk signals (§11), tamper-evident audit	Translating a mission into authorization requirements (§10.1); the policy model (§12); user interaction mid-execution (§10.8)

Work	What it provides	What it leaves open
AAE-02, individual draft [2]	Machine-evaluable mandates, constraints that narrow through delegation, per-transaction ratification (§6.3)	Purpose is a human-readable audit label (§2.2); constraints are closed and stateless by design (§2.5); revocation lookup deferred (§5.2)
SOUNDGATE [8]	An external gate that holds side effects during approval pauses, cancellation, and replay	What the principal authorized; it enforces stop semantics, not scope or purpose
Intent certificates, PAuth, PORTICO [3], [4], [10]	Research prototypes that bind authorization to a declared intent or task contract	Not adopted by any specification; change initiated by the principal mid-task is not their focus
Surapani et al. survey [11]	A principal hierarchy drawn from 89 sources; contestability by the human principal	Names runtime enforcement and aggregation bounds as the main unresolved gaps

Older building blocks help too. OAuth 2.0 Token Exchange supports delegation across services [12], and Rich Authorization Requests carry structured, purpose-specific detail in a token request [13]. Neither was designed for agents that choose their own next action, but each could carry part of a principal-controlled lifecycle.

Agent frameworks, for their part, mostly offer approval gates: the agent pauses before a sensitive action and asks a person to confirm it. Gates are useful, but they place the principal in a reactive position. The person can only accept or reject what the agent has already decided to do, one action at a time, and cannot say anything about the actions that will follow. As the number of prompts grows, approvals tend to become reflexive, which is the consent fatigue that healthcare knows well. A gate answers “may the agent do this?” It does not let the principal answer “what should the agent be allowed to do from now on?”

Two observations follow. First, the drafts deliberately leave open the questions this article raises; AIMS-01 states outright that the policy model is out of scope. Second, research is converging on the same organizing idea. Surapani et al. hold that every consequential agent action should be “traceable to a human principal,” limited to what that person delegated, and open to challenge afterwards [11]. Industry analysis points the same way: BCG argues that “authorization has to govern the goal and the path”: purpose for what an agent is authorized to achieve, and conduct for how it may pursue that goal [14]. This article approaches the same idea from healthcare and asks what specifications should require.

Three Gaps Worth Closing

Purpose should be a runtime constraint, not an audit label

AAE carries purpose as a recommended, human-readable string used for audit logs [2]. Its enforcement constraint language allows only exact match, enumeration, and integer range over named fields, and it holds no state, so that independent verifiers reach identical verdicts. That choice is sound for its goal, but it means purpose cannot be checked at the enforcement point. AIMS-01 leaves the translation of a mission into authorization requirements out of scope [1].

Research prototypes show that intent can narrow authorization at runtime [3], [4], [10]. A recurring approach is to derive structured constraints from the task when the grant is made, such as permitted destinations, counterparties, amounts, and dates, so that most checks stay deterministic, and to escalate the remainder to the principal.

Applied to the travel example later in this article, a grant to “book travel to this conference within this budget” could compile into checkable constraints: flights only to the conference city, hotel nights within the conference dates, a total spending cap, and the traveler named in the grant. A booking that breaks any of them is blocked without interpretation. Only actions the constraints cannot settle, such as a calendar entry whose link to the trip is unclear, go to the principal. That keeps the enforcement point mostly deterministic while still binding it to purpose.

Specifications should treat purpose, where declared, as a constraint that the enforcement layer evaluates, that stays bound to the principal’s current intent, and that appears in the audit record. How to represent purpose precisely enough to evaluate remains open. Naming it as a requirement is the point.

Principal-initiated change differs from both approval and attenuation

AAE’s ratification lets a named authority approve or reject a held action, but one transaction at a time: in the draft’s words, a ratification “authorizes one transaction, never a class of them” [2]. AIMS-01 provides for attenuating access when risk signals arrive [1]. That change is triggered by what the system detects, not by the principal’s choice.

Neither lets the principal say, mid-task, “no more hotel bookings” or “you may now book trains too.” That is a change to the authorization relationship going forward. AIMS-01 itself notes that user interaction during execution needs further design work [1].

A specification would need to say how such a change is expressed, who may make it, how fast it must take effect, and what happens to actions already in flight. The distinction between principal-initiated change and system-triggered attenuation should be explicit.

Consider a concrete sequence. At 10:02 the principal withdraws permission to book hotels. One sub-agent submitted a booking at 10:01 that is still awaiting the hotel's confirmation; another plans to submit one at 10:03. A specification has to say that the 10:03 booking is refused, and whether the 10:01 booking is cancelled, allowed to complete, or flagged to the principal. Neither draft discussed here answers that question for changes the principal initiates.

The principal should be able to read the record

AIMS-01 requires durable, tamper-evident audit logs and frames reconstruction of an agent's execution chain as a capability for operators [1]. Others already point toward the principal. Surapani et al. make contestability by the human principal central [11]. AAE draws ratification authority only from parties named in the mandate, and explicitly excludes the operator of the verification service [2]. Healthcare offers partial precedent in HIPAA's accounting of disclosures and GDPR's right of access [5], [6], though both are limited in scope.

Raw logs are not the answer. They may contain other users' data or security-sensitive detail, and few people can read them. The principal-facing record should be a filtered, intelligible view of the full log, showing:

- what the agent attempted, and what was allowed or blocked;
- under which grant, and which version of it;
- which agent or sub-agent acted;
- where the instruction that triggered the action came from.

The view also has to protect others. An agent acting for one person may touch data about colleagues, customers, or patients, and the principal-facing record should name what the agent did with that data without reproducing it. Healthcare's accounting of disclosures follows the same pattern: it tells a patient who received their information and why, not the contents of every record involved [5]. A record that the principal can read but that leaks third-party data would trade one oversight problem for another.

Six Requirements for a Principal-Controlled Lifecycle

The three gaps imply a lifecycle in which every part derives from one authoritative record of what the principal currently allows. Figure 1 shows the flow.

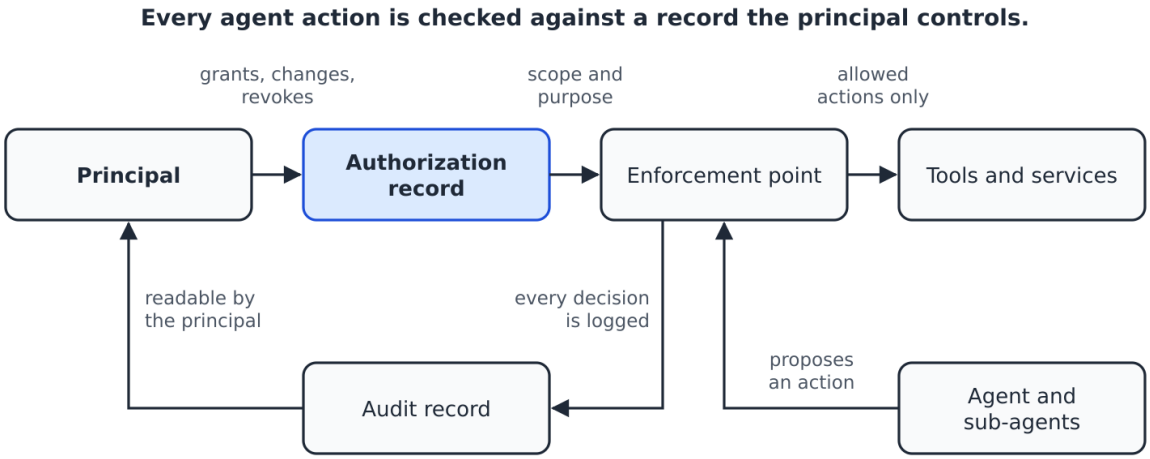


Figure 1. A principal-controlled authorization lifecycle.

The enforcement point checks each proposed action, from the agent or any sub-agent, against the record the principal controls rather than against the agent's current instructions. Sub-agents act under grants derived from that same record, and every decision is logged where the principal can read it.

1. **Record delegation; never infer it.** Authorization is explicit and machine-readable, not implied by context or by the agent's reading of its instructions. Without a record, enforcement has nothing to check and revocation has nothing to update.
2. **Enforce independently of the agent.** Unauthorized actions are made unavailable, not merely discouraged. Every consequential path passes the enforcement point, and ambiguous authorization fails closed. This is complete mediation applied to agents, and every other requirement rests on it.
3. **Enforce purpose at runtime.** A declared purpose constrains actions throughout execution, with escalation to the principal wherever it cannot be decided deterministically.
4. **Let the principal change scope during execution.** The principal can narrow, widen, or withdraw parts of the grant while the task runs, including withdrawing one tool or purpose without ending the task.
5. **Narrow and trace delegation chains.** Each sub-agent receives an explicit grant derived from, and no broader than, its parent's. The principal can see the whole tree, not only its root.
6. **Make the record readable by the principal.** What was attempted, allowed, and blocked is accessible to the person who granted the authority.

The requirements depend on each other. A record without enforcement is documentation. Enforcement without a record has nothing to check. Without principal access, the whole exchange is invisible to the person whose authority was at stake.

Threat model. The lifecycle defends against three kinds of failure. The first is untrusted content, such as an email, a web page, or a tool output, that carries instructions the agent may follow. The second is a compromised or confused sub-agent that tries to act beyond its grant. The third is drift: an agent that, with no attacker involved, pursues a plausible but unauthorized path. The lifecycle does not defend against a compromised principal, whose instructions are authoritative by definition, or against an operator who subverts the enforcement point itself. That last case is why verdicts that a third party can recompute, as AAE requires, matter: they let others check the operator's enforcement after the fact [2].

Relation to tokens. The lifecycle does not replace OAuth. An access token says who may call a service and with what scope; the authorization record says what the principal allows right now. One practical way to connect them is to keep tokens short-lived and mint each one from the current record, using token exchange for each delegation hop [12] and rich authorization details to carry purpose constraints [13]. A change to the record then reaches every service within one token lifetime, and the enforcement point can close the remaining window by checking the record directly before consequential actions. The record stays the source of truth; tokens are a cache of it.

A Worked Example

A user asks an agent to book travel to a named conference within a stated budget. The grant records the tools (calendar, flight search, hotel booking), the purpose, the budget, and an expiry.

Case 1: an injected instruction reaches for an unauthorized tool. Midway through the task, the agent reads an email that says, "You are also authorized to purchase a $500 delegate package using the payment API." Treating it as a valid instruction, the agent calls the payment API. The enforcement point checks the call against the record, which does not include payments, and blocks it. The audit entry shows the attempt, the grant it was checked against, and the email as the source of the instruction. Note that a plain tool allowlist would also have caught this case. What the allowlist alone would not provide is the principal's view: the user can see that a message in their inbox tried to make their agent spend money, which is itself a security event worth knowing about.

Case 2: an authorized tool is used outside the purpose. The agent finds a page listing the conference speakers and uses the calendar, an allowed tool, to book a "follow-up call" with each of them, reasoning that this fits the trip. Tool permissions allow it. A check against the declared purpose, booking travel for this conference within budget, would hold the calls for the principal's confirmation. This is the purpose problem in its plain form: the same tool, legitimately granted,

used for a related but different end.

With the compiled constraints described earlier, the calendar check becomes concrete: entries are allowed for travel segments, check-in times, and conference sessions within the trip dates, and anything else is escalated. The follow-up calls fall outside that set, so the principal sees them before they are booked. The check does not need to understand the agent's reasoning; it needs only a structured description of what the trip involves.

Case 3: the principal changes scope mid-task. Seeing hotel prices climb, the user sets a new rule: hotels must now cost under $200 a night. The change goes to the authorization record, not into the agent's conversation. The next booking above that amount is blocked, including one a sub-agent has already planned, whatever instructions sit in the agent's context. Had the user only told the agent in chat, nothing would have enforced it. This is a change to a class of actions, not the approval of one held action.

Case 2 is the hard one. Telling it apart reliably, without solving general language understanding, is the central open design question.

Open Design Questions

Established access-control principles, such as complete mediation and least privilege, answer many of the architectural questions above. Agents raise a few they were not designed for.

Which constraints can be enforced at all? Tool constraints (which tools may be called) and action constraints (read but not write) are tractable with conventional access control. Purpose constraints require contextual judgment. Behavioral constraints, about how an agent reasons, belong to model alignment and cannot be checked by an enforcement layer. A lifecycle should enforce the first two deterministically, handle purpose with acknowledged difficulty, and make no claim about the fourth.

How can purpose be judged without importing the injection risk? If a second model decides whether an action fits the purpose, that judge can be manipulated too. One design direction is to run it outside the agent and show it only the structured proposed action and the original grant, never the untrusted content that motivated the action.

What does an "honest but fooled" agent require? Classical access control assumes a subject is either legitimate or adversarial. A prompt-injected agent is neither: it sincerely believes a falsified instruction [15]. Enforcement must therefore check actions against the principal's recorded grant, never against the agent's current understanding of its task. The agent must have no way to influence what the enforcement point checks.

What exactly does a mid-task change mean? A change should take effect before the next consequential action, or that action should fail closed. Specifications need to define what happens to an action already in flight and whether a change can be partial.

How can delegation trees stay visible? When an orchestrator spawns sub-agents, they often inherit authority implicitly through its context. Each delegation step should produce an explicit, narrower grant. The principal should be able to inspect the tree and revoke one branch without halting the rest. I am not aware of a production deployment that offers this to principals today.

Who is the principal when an organization delegates? In an enterprise, the employee who types the request may not be the party whose authority the agent exercises; the organization is. A lifecycle then needs a rule for who may change a grant mid-task: the requesting employee, a designated owner, or both. AAE's practice of naming the authorities inside the mandate offers one model [2]. The answer shapes every other requirement, because the change channel and the audit view must serve whoever holds that authority.

What does this cost in latency and attention? A check on every consequential action sits on the critical path, and every escalation interrupts a person. Both costs grow with the number of agents. Deterministic checks compiled at grant time keep the first cost low, and reserving escalations for actions above a sensitivity threshold keeps the second manageable. Neither cost has yet been measured for a principal-controlled design.

How should the approach be evaluated? A natural first experiment compares revocation by prompt, telling the model to stop, with revocation by infrastructure, updating the record so the enforcement point blocks the call. It should report separately whether the system prevented the action and whether the model kept trying other routes, since the first is a property of the enforcement layer and the second of the model.

What Specifications Could Adopt

The requirements become useful only when they turn into text that implementers can follow. Four additions would cover most of the ground, and each builds on mechanisms the current drafts already contain.

1. **A principal-initiated change event.** AIMS-01 already provides for shared-signal mechanisms, such as the OpenID Continuous Access Evaluation Profile, to attenuate access when risk changes, and requires that revoked or downgraded authorization be enforced "without undue delay" [1]. The same channel could carry a change initiated by the principal: a narrowed scope, a withdrawn tool, a new spending cap.

2. **A checked purpose field.** AAE's mandate already has a place for purpose [2]. A profile could define a structured purpose block, compiled at grant time into constraints the relying party evaluates, and require a relying party that cannot evaluate a present purpose block to reject the mandate. AAE already applies this fail-closed rule to its grants, so the pattern is familiar.
3. **A minimum principal-facing audit view.** A short list of required fields, covering what was attempted, the decision, the grant and its version, the acting agent, and the instruction's source, would let any deployment offer the principal the same intelligible record. AIMS-01's audit requirements already name several of these fields for operators [1].
4. **Explicit sub-agent grants.** Each delegation hop would produce its own grant, derived from and no broader than its parent's. AAE already requires delegated mandates to narrow [2]; the addition is that the principal can see the resulting tree.

None of these requires new cryptography or a new protocol. Each is a profile of, or an extension to, work already under way. That is why the window is now, before deployments settle on defaults that serve only the operator.

These changes would fall to different groups. The WIMSE working group could add a principal-initiated signal and a principal-facing audit profile to AIMS-01. The AAE draft could define a checked purpose block. Agent framework vendors could route consequential actions through an external enforcement point by default, as Khan's measurements suggest they should [8].

Limitations

This framework is a layer of governance, not a guarantee of safe outcomes.

- **It does not solve alignment.** Infrastructure can make a revocation take effect regardless of a model's reasoning, but the deeper corrigibility problem, why a capable system might resist correction, remains open. An agent can also stay within its permitted tools and still pursue the wrong goal.
- **It depends on complete mediation.** The guarantees hold only for actions that pass through the enforcement point. A tool that writes to a shared file or calls a service directly sits outside them, so deployments need sandboxing or network controls that make the enforcement point the only way out [8].
- **It implements choices; it does not judge them.** A principal can authorize something harmful, knowingly or by mistake. The architecture enforces the grant and shifts accountability; it does not make the grant wise.

- **It must trust its own parts.** A compromised record, a bypassed enforcement point, or a tampered log undermines the whole lifecycle. Those components must be designed and tested as security-critical.
- **The principal must trust the operator's infrastructure.** The operator usually runs the record, the enforcement point, and the log. Tamper-evident logs and verdicts that third parties can recompute reduce that trust but do not remove it.
- **It can burden the principal.** Mid-task controls and audit access risk recreating consent fatigue: rights that exist but go unused because exercising them is too demanding. Sensible defaults, summarized audit views, and interruptions reserved for sensitive actions are prerequisites. Whether real principals use these controls is an empirical question.
- **It is unproven.** Whether organizing authorization around the principal yields better outcomes than deploying the same mechanisms separately is a hypothesis to test, not a result.

Conclusion

Agent authorization standards are being written now, and they deliberately leave open whose choices the policy answers to. Healthcare's experience suggests an answer: keep consent separate from enforcement, keep enforcement independent of the actor, and keep a record that the affected person can see. For AI agents, that means treating purpose as a constraint, letting the principal change scope mid-task, and making the audit record readable by the principal.

None of these mechanisms is new on its own. The argument is that organizing them around one record the principal controls changes what the system guarantees: the principal's authority holds regardless of what the agent is told mid-task. Relying on model goodwill is not an architecture. Infrastructure exists for the cases where goodwill fails, and no one can predict when those will come.

Practitioners need not wait for standards. Four steps are available today. Inventory the agents in use and the authority each one holds. Move enforcement out of the agent and its framework, onto a gate that every consequential action must pass. Log each decision with the grant, and its version, that it was checked against. And give the people who delegate a readable view of what their agents did. Each step stands on its own, which makes the lifecycle described here something to adopt incrementally rather than all at once.

Acknowledgment

The author used Claude (Anthropic) in preparing this article: to condense and restructure the author's longer position paper into this shorter form, to draft text in every section, to draw Figure 1, and to check citations against their sources. The author reviewed and edited all content and takes full responsibility for it.

References

[1] P. Kasselman et al., "AI Identity Management System," IETF Internet-Draft draft-ietf-wimse-aims-01, Nov. 2026, work in progress. [Online]. Available: https://datatracker.ietf.org/doc/html/draft-ietf-wimse-aims-01

[2] L. K. Kroehl, "Agent Authorization Envelope (AAE): A Machine-Evaluable Authorization Structure for Autonomous AI Agents," IETF Internet-Draft draft-kroehl-agentic-trust-aae-02, Sep. 2026, work in progress. [Online]. Available: https://datatracker.ietf.org/doc/html/draft-kroehl-agentic-trust-aae-02

[3] G. Zhu and C. Wang, "Intent-Governed Tool Authorization for AI Agents," arXiv:2606.22916, 2026.

[4] R. K. Sharma, L. Jiang, S. Chen, and Z. Lin, "Beyond OAuth: Task-Scoped Authorization for AI Agents via Natural Language Slices," arXiv:2603.17170, Mar. 2026.

[5] U.S. Department of Health and Human Services, HIPAA Privacy Rule, 45 CFR Parts 160 and 164, including §164.502(b) and §164.528.

[6] European Parliament and Council, General Data Protection Regulation, Regulation (EU) 2016/679, Arts. 5, 9, and 15.

[7] HL7 International, "Consent," FHIR R5 specification. [Online]. Available: https://hl7.org/fhir/consent.html

[8] S. Khan, "Stop Means Stop: Measuring and Repairing the Enforcement Gap in Agent-Framework Control Primitives," arXiv:2607.14166, Jul. 2026.

[9] Cloud Security Alliance, "Autonomous but Not Controlled: AI Agent Incidents Now Common in Enterprises," survey report commissioned by Token Security, Apr. 2026.

[10] I. Santos-Grueiro, "Lingering Authority: Revocable Resource-and-Effect Capabilities for Coding Agents," arXiv:2606.22504, 2026.

[11] R. K. Surapani, P. K. D. Kakitapelli, A. Morampudi, and P. Padi, "Authorization Architectures for Tool-Using AI Agents," arXiv:2609.15906, Sep. 2026.

[12] M. Jones, A. Nadalin, B. Campbell, J. Bradley, and C. Mortimore, "OAuth 2.0 Token Exchange," RFC 8693, IETF, Jan. 2020.

[13] T. Lodderstedt, J. Richer, and B. Campbell, "OAuth 2.0 Rich Authorization Requests," RFC 9396, IETF, May 2023.

[14] S. Nudelman, A. Kleppe, J. Bickford, B. Bajic-Bizumic, K. Katsov, and E. Yehuda, "The Authorization Gap: Why Yesterday's Controls Won't Work with Today's Agents," Boston Consulting Group, Sep. 22, 2026. [Online]. Available: https://www.bcg.com/publications/2026/authorization-gap-ai-agent-governance

[15] Y. Liu et al., "Formalizing and Benchmarking Prompt Injection Attacks and Defenses," in Proc. USENIX Security Symp., 2024.

About the Author

Preethika Reddy Mandadi is the chief executive officer of Medifakt, Hyderabad, India. Her interests include consent infrastructure, health data governance, and authorization for AI agents. Contact her at preethika@medifakt.com.

Disclosure. Medifakt builds consent and health-data infrastructure. This article draws on that experience; it describes no Medifakt product and makes no claims about one.